SPEC_U6 - Mantenimiento de Food Store

Integrantes: Exequiel Arancibia y Joaquin Romero. PostgreSQL 17; Windows.

Alcance y reglas

Continuar la base acumulada U1-U5 en una copia privada llamada `foodstore`. Conservar usuario (equivalente a cliente de la guía), catálogo, pedidos, detalle, índices, vistas, materializadas y extensión FNBC. El origen U5 se conserva. El ensayo sólo crea `foodstore_restore_test`; abortar si ya existe. No eliminar ni reemplazar foodstore; no usar DROP, --clean ni --create al restaurar. Operar en loopback con pg_dump/pg_restore 17 y autenticación SCRAM. Las claves se obtienen mediante PGPASSFILE privado con ACL restringida, nunca desde el código, argv, logs entregados ni adjuntos de IA.

Tarea	Frecuencia propuesta	Condición de éxito y evidencia
Respaldo custom	Diaria, 02:00	pg_dump sale 0, archivo fechado no vacío, TOC legible y log; publicar .dump sólo al completar; conservar SHA-256.
Salud	Diaria, 02:15	Ejecutar health_check.sql con ON_ERROR_STOP: tamaño, conexiones, tablas y consultas activas.
Mantenimiento	Semanal, domingo 03:00	Revisar autovacuum/ANALYZE y tuplas muertas; justificar ANALYZE puntual por la observación de la copia. Sin VACUUM FULL periódico.
Restauración de prueba	Semanal, domingo 03:30	Base nueva validada: cinco conteos iguales a la línea base, productos/detalles inválidos 0; fallo implica salida no cero.

Medición y aceptación

RPO previsto <=24 h con respaldo diario exitoso; RTO previsto <=120 s en este laboratorio. Medir RTO con reloj monotónico inmediatamente antes de createdb y hasta finalizar las validaciones. Antigüedad observada al inicio del ensayo desde el inicio del respaldo: referencia conservadora del corte, no prueba de pérdida real ni recuperación a un instante (PITR). Registrar horarios UTC y UTC-03, tiempos, tamaños, hashes y resultados. Comparar además huellas completas de las tablas para verificar conservación. La programación es propuesta: primero ejecutar manualmente y comprobar fallos.

Entrega y revisión

Un único informe_u6.pdf con esta especificación (<=1 página), informe post-incidente (<=1 página), evidencias y anexos con fuentes completos. Incluir fuentes extraíbles como adjuntos del PDF; omitir dump y datos privados. OpenCode recibe SPEC y AGENTS sin datos; conservar prompt/respuesta, revisar cada comando y probar antes de aceptar. Distinguir revisión automática de la intervención humana. La restauración sin owners/ACL no certifica recuperar la política de roles U5: la copia descartable no se habilita a la aplicación.

Informe post-incidente - Simulacro U6

Exequiel Arancibia y Joaquin Romero | Base de Datos II | 03/11/2026

Respaldo utilizado

Base principal `foodstore`, continuadora de U1-U5; PostgreSQL 17.11 y PowerShell 7.6.5. Se utilizó el último manifiesto SUCCESS, cuyo respaldo custom comenzó el **03/11/2026 a las 18:56:43,475 UTC-03** (21:56:43,475 UTC) y se completó a las 18:56:47,637 UTC-03. El archivo fechado ocupa **8.224.005 bytes**; se verificaron TOC y SHA-256. La evidencia incluye nombre completo, log y manifiesto sin credenciales.

Recuperación y objetivos

El ensayo comenzó a las **18:57:15,284 UTC-03**. El cronómetro monotónico abarcó creación de `foodstore_restore_test`, aislamiento, restauración, validación y comparación; no se eliminó ni reemplazó `foodstore`.

Indicador	Previsto para el laboratorio	Observado
RTO: tiempo de recuperación	<=120 s	**11,350 s**, objetivo cumplido.
RPO: antigüedad de la copia de referencia	<=24 h, con respaldo diario exitoso	**31,808 s** al iniciar el ensayo.

El RPO observado usa el inicio de pg_dump como referencia conservadora del corte, no la hora final del archivo. No se midió pérdida real ni se probó PITR; las lecturas previas/posteriores coincidieron en una ventana sin escrituras.

Validaciones y conservación

Coincidieron categorías **2**, usuarios **20.001**, productos **50.000**, pedidos **225.000** y detalles **499.997**; productos y detalles inválidos: **0/0**. Las cinco huellas completas coincidieron. El control complementario posterior confirmó las 12 tablas base, definiciones heredadas y ambas materializadas; la principal y el origen U5 permanecieron intactos. El destino queda aislado: `--no-owner --no-privileges` no recupera la política de roles/ACL de U5.

Problema, mejora y asistencia

Se detectó que la limpieza del entorno dejaba variables libpq vacías y provocaba un fallo de conexión; se corrigió con eliminación efectiva de las variables y se repitieron las pruebas. El fallo real controlado de pg_dump devolvió error sin publicar copia válida; repetir restauración también abortó. Mejora propuesta: mantener otra copia cifrada fuera del equipo y verificarla periódicamente en un servidor independiente.

OpenCode generó la propuesta a partir de SPEC/AGENTS autorizados. La revisión técnica de Codex corrigió, entre otros puntos, una opción inexistente de createdb; luego se ejecutaron y verificaron los scripts. La persona solicitante revisó las conclusiones y confirmó la entrega el 03/11/2026. Esa conformidad se distingue de las pruebas automáticas y no implica lectura exhaustiva del código.

Resolución y evidencias - Unidad 6

Exequiel Arancibia y Joaquin Romero. Base de Datos II, ciclo 2026. Todas las medidas de este informe proceden del laboratorio ejecutado el 03/11/2026; no son estimaciones de una salida que se esperaba obtener.

1. Continuidad del proyecto y alcance

Se retomó la base acumulada de los TP1-TP5 y las Unidades 4 y 5. En el servidor privado se creó `foodstore` como copia de la base U5, conservando datos, propietarios y objetos. Se mantuvo el origen U5 sin reemplazarlo. La copia conserva el catálogo, baja lógica, precios históricos, subtotal GENERATED, reglas de pedidos de TP2, índices y vistas de TP5, FNBC y materializada diaria de U4, además de las funciones y datos_anon de U5.

La guía llama `cliente` a la entidad que nuestro modelo implementa como `public.usuario`. Por eso se adapta esa referencia a usuario y se conservan sus 20.000 registros heredados más la cuenta sintética U5. No se creó una tabla ficticia cliente ni se reemplazó el proyecto por un ejemplo mínimo.

Objeto principal	Filas exactas antes del respaldo
public.categoria	2
public.usuario, equivalente a cliente	20.001
public.producto	50.000
public.pedido	225.000
public.detalle_pedido	499.997

El entorno observado es PostgreSQL **17.11**, PowerShell **7.6.5**, Windows, `127.0.0.1:55605`. Cumple PostgreSQL 16 o superior. Las versiones finales de los scripts requieren PowerShell 7.5+ por el manejo explícito de fechas JSON. Las operaciones se realizaron con el administrador técnico del laboratorio; no se ampliaron permisos de los roles de aplicación.

2. Parte 1 - Especificación y frecuencia

La primera página contiene SPEC_U6 íntegra, con las cuatro tareas exigidas, frecuencias y criterios ejecutables de éxito. Los horarios propuestos se interpretan en UTC-03. Son una propuesta de programación: no se instaló una tarea recurrente ni se presenta una ejecución manual como si fuera un cron.

Se propone respaldo diario 02:00, salud diaria 02:15, revisión semanal domingo 03:00 y restauración semanal domingo 03:30. Un error requiere investigación y no se considera una copia válida para cumplir el RPO diario. Los objetivos previstos antes de las pruebas son RPO <=24 h y RTO <=120 s para este equipo de laboratorio.

3. Parte 2 - Respaldo automatizable probado

El script backup_foodstore.ps1 se ejecutó manualmente, con su dependencia tools/ps_common.ps1. Sigue este procedimiento:

1. Comprueba loopback, versión 17, operador autorizado, rutas privadas, PGPASSFILE protegido y confirmación de una ventana sin escrituras.
2. Ejecuta validaciones.sql y guarda una línea base coherente de conteos, huellas y controles lógicos. No imprime registros personales.
3. Inicia pg_dump -Fc hacia un nombre UTC único terminado en .partial. Comprueba inmediatamente el código nativo de salida.
4. Verifica tamaño no vacío y lectura del TOC con pg_restore --list; repite la línea base y exige igualdad para detectar cambios observables.
5. Calcula SHA-256 y publica el archivo .dump sin sobrescritura. Sólo después publica el manifiesto SUCCESS, también sin sobrescribir otro archivo.
6. Guarda el registro en la carpeta logs. Si una fase falla, devuelve código distinto de cero; un parcial o un dump sin manifiesto no se elige como válido.

Las claves no aparecen en el código, argumentos, PDF ni adjuntos. Se usa PGPASSFILE externo con ACL restringida al operador, SYSTEM y Administradores. Se eliminan variables libpq que podrían redirigir una conexión,

y se utiliza -w para evitar una solicitud interactiva de contraseña. Estas medidas no sustituyen un diseño de cuentas de producción con responsabilidades separadas.

Evidencia del respaldo real

Campo observado	Resultado
Archivo	foodstore_20261103T2156409509091Z_fd85fddaeb454d6e8aaa06735a9d316d.dump
Formato	Custom, generado con -Fc; TOC leído correctamente.
Inicio de pg_dump	03/11/2026 21:56:43,475 UTC = 18:56:43,475 UTC-03.
Finalización y publicación	21:56:47,637 UTC = 18:56:47,637 UTC-03.
Tamaño	8.224.005 bytes, aproximadamente 7,843 MiB.
Estado y salida del script	SUCCESS, código 0.
SHA-256	`b7909e09d63c4a84fca707a7774020d725a074fdd6efa54bd8ec19836af9dfb3`

El nombre se fecha al iniciar el trabajo; el manifiesto distingue ese momento del inicio posterior de pg_dump. Para discutir el RPO se usa este último, no la fecha del nombre ni la última modificación del archivo.

Fragmento real del log de éxito:

```
SUCCESS backup archivo=foodstore_20261103T2156409509091Z_fd85fddaeb454d6e8aaa06735a9d316d.dump
bytes=8224005 sha256=b7909e09d63c4a84fca707a7774020d725a074fdd6efa54bd8ec19836af9dfb3
```

La línea se distribuye visualmente para legibilidad; el log íntegro y el manifiesto se incluyen como adjuntos de evidencia. El .dump permanece privado: la plataforma solicita un único PDF y el archivo contiene datos y material de autenticación almacenado. Se aporta evidencia de su creación, tamaño, integridad y restauración efectiva, sin inventar un límite de tamaño del aula.

Control de error real

Se denegó temporalmente la creación de archivos únicamente en el directorio vacío de backups de esta copia. pg_dump falló realmente y el script devolvió código **1**, sin .dump completado ni manifiesto SUCCESS. La ACL original se restauró en finally y después se ejecutó el respaldo exitoso anterior. Fue una prueba deliberada del manejo de errores, no un incidente de producción.

4. Parte 3 - Salud y mantenimiento

health_check.sql se ejecutó con psql -X y ON_ERROR_STOP. Sus cuatro controles son consultas de sólo lectura; el resultado se conservó completo.

Control	Observación real e interpretación
Tamaño de foodstore	171.054.771 bytes en la observación anterior al mantenimiento.
Conexiones/transacciones	Se consultaron numbackends, commits, rollbacks, deadlocks y reinicio de contadores de la base. Son acumulados, no tasas ni ventas.
Estado de tablas	Se obtuvieron estimaciones de 14 relaciones estadísticas, incluyendo las materializadas, y sus históricos de vacuum/ANALYZE.
Consultas activas	No había otras consultas activas al tomar la muestra; cero es una salida válida. Se excluye el chequeo propio y no se publica SQL crudo, direcciones ni identidades personales.

La copia recién creada tenía n_live_tup=0 y ambos históricos de ANALYZE en NULL para pedido/detalle, pese a los conteos reales de 225.000 y 499.997. Esto no significa que estuvieran vacías ni prueba ausencia de pg_statistic. Autovacuum y track_counts estaban **on**; no se desactivaron.

maintenance.sql observó configuración, opciones por tabla, estimaciones y conteos reales antes de decidir. Aplicó un **ANALYZE puntual** a esas dos tablas, sólo cuando no había histórico de análisis manual ni automático, para establecer una línea base documentada de la copia.

Tabla	n_live_tup antes	Después	n_dead_tup después	last_analyze observado, UTC-03
pedido	0	225.000	4	03/11/2026 18:40:10,232
detalle_pedido	0	499.997	2	03/11/2026 18:40:10,302

Son estimaciones estadísticas, comparadas separadamente con COUNT(*). El análisis registra un contador manual por tabla; no modifica sus filas. No se ejecutó VACUUM FULL ni se justificó una reescritura por estas pocas tuplas muertas estimadas. ANALYZE actualiza estadísticas del planificador; VACUUM ordinario tiene otros fines y no equivale a VACUUM FULL. Mantenimiento rutinario de PostgreSQL 17.

5. Parte 4 - Recuperación contra reloj

restore_test.ps1 valida primero los clientes, endpoint, permisos del operador, ausencia del destino y manifiesto exitoso más reciente. Rechaza archivos fuera de las rutas admitidas, verifica tamaño/SHA-256 y comprueba la línea base. Ese preflight no forma parte del intervalo definido por la consigna.

Inmediatamente antes de createdb inicia Stopwatch. A continuación:

1. Crea exclusivamente foodstore_restore_test desde template0.
2. Limita conexiones a cero para no administradores y revoca CONNECT/TEMP de PUBLIC y los seis roles de negocio. Comprueba que no haya otra sesión.
3. Restaura el archivo con --no-owner --no-privileges --exit-on-error, sin --create, --clean ni DROP.
4. Cierra acceso a esquemas y rutinas restauradas y verifica permisos efectivos.
5. Ejecuta validaciones.sql en un snapshot de sólo lectura; compara explícitamente todos los conteos, huellas y controles lógicos.
6. Detiene el cronómetro sólo tras esas comprobaciones exitosas y registra el resultado. Un error o comparación falsa nunca produce SUCCESS.

createdb y el cierre de permisos son comandos consecutivos, no atómicos: se usó un servidor privado en loopback con ventana exclusiva de operadores. La copia descartable no se habilitó a la aplicación. Restaurar sin owners/ACL cambia propietarios de funciones DEFINER y no reconstruye la seguridad U5; esa diferencia es intencional y queda acotada por el aislamiento. Opciones de pg_restore 17.

Resultados mínimos: línea base y restauración

Tabla real	Antes del respaldo	Restaurada	Coincidencia
categoria	2	2	Sí
usuario, equivalente a cliente	20.001	20.001	Sí
producto	50.000	50.000	Sí
pedido	225.000	225.000	Sí
detalle_pedido	499.997	499.997	Sí

Restricción lógica	Resultado esperado	Resultado restaurado
producto.precio <0 o stock <0	0	0
detalle.cantidad <=0 o precio_unitario <0	0	0
Huellas completas distintas en las cinco tablas	0	0

Los campos examinados son NOT NULL en el modelo heredado. Las huellas son checksums MD5 de filas completas ordenadas y no hashes de contraseñas, pruebas de anonimato ni firmas criptográficas de autenticidad. El archivo de respaldo se verifica con SHA-256, con una finalidad diferente.

RTO y RPO observados

Medida	Valor real	Evaluación
Inicio del ensayo	03/11/2026 21:57:15,284 UTC / 18:57:15,284 UTC-03	Antes de createdb.
RTO monotónico	**11,3500773 segundos**	Menor a 120 s. Incluye creación, aislamiento, restore y comparaciones.
Antigüedad de referencia al iniciar	**31,8083982 segundos**	Menor a 24 h.
Hora del registro final	21:57:26,676 UTC	Posterior al stop y a escribir el log; no se usa su diferencia como sustituto del cronómetro.

RTO es duración de recuperación. RPO expresa cuánto retrocede la información recuperable: aquí se discute mediante la antigüedad del respaldo pedida por la guía. La referencia temporal es el inicio registrado de pg_dump, anterior a su snapshot interno; es una cota conservadora, no una marca exacta de snapshot exportado. Las dos lecturas de línea base no comparten automáticamente ese snapshot y se trabajó sin escritores; sus datos coincidieron.

No hubo borrado real de la principal ni se midieron transacciones perdidas. Un dump diario no proporciona por sí solo recuperación a un instante mediante WAL/PITR. Estos resultados son de este equipo y ensayo; los objetivos propuestos no se presentan como garantía de un servicio productivo. pg_dump 17 y copias consistentes.

Controles complementarios posteriores

Fuera del intervalo RTO se compararon **las 12 tablas base** y sus huellas, 22 relaciones inventariadas, 10 funciones públicas, 25 índices, el inventario de 35 restricciones, 15 triggers y definiciones de vistas: coincidieron. Ambas materializadas también coincidieron: mensual con 14 filas y diaria con 286, verificadas por contenido completo. La principal foodstore y el origen U5 conservaron sus datos y propietarios. Estos controles adicionales no sustituyen ni alteran la medición mínima del cronómetro.

La línea base ordena las huellas por PK y el postflight por representación JSON: sólo se comparan fuente y destino dentro del mismo método, no los valores entre ambos archivos. El inventario de restricciones verifica nombre, tipo y estado validado; no se presenta como una revisión semántica exhaustiva de cada definición de restricción.

Finalmente, se volvió a invocar el script con foodstore_restore_test existente: devolvió código **1** en preflight, sin crear, eliminar ni sobrescribir la base.

6. Parte 5 - Informe y verificación de la asistencia

La segunda página es el informe post-incidente de una página como máximo. La especificación también ocupa una sola página. El resto de este documento son evidencias y anexos, separados de esos dos límites.

OpenCode **1.18.23**, modelo openai/gpt-6.1-sol, recibió únicamente SPEC_U6 y AGENTS, con autorización explícita. Generó una propuesta sin herramientas ni ejecución. Codex revisó e integró los scripts, contrastó las consultas y ejecutó las pruebas. No se atribuye esa revisión automática a una persona.

Hallazgo real	Corrección y comprobación
La propuesta incluía --connection-limit=0 en createdb.	La ayuda real de PostgreSQL 17 demuestra que no admite esa opción. Se quitó y se aplicó ALTER DATABASE inmediatamente después; el ensayo completo pasó.
El manejo JSON convertía fechas ISO a DateTime antes de ParseExact.	Se preservan cadenas con -DateKind String y se declara PowerShell 7.5+. El manifiesto real fue leído y la recuperación pasó.
Una integración de Codex dejó variables libpq vacías con SetEnvironmentVariable.	Se detectó service file vacío en el preflight, se sustituyó por Remove-Item Env: y se probó su eliminación efectiva. No se culpa a OpenCode por este fallo.

Hallazgo real	Corrección y comprobación
La salida propuesta no separaba una carpeta logs y difería del contrato de huellas.	Se implementaron backups/logs, cinco MD5 de 32 caracteres y dos controles lógicos; SHA-256 verifica el dump.

El error de sintaxis ${Name} también se corrigió: la propia respuesta ya lo advertía, por lo que no se presenta como un descubrimiento independiente. Prompt, respuesta íntegra, metadatos y revisión quedan adjuntos al PDF. La persona solicitante revisó las conclusiones y confirmó expresamente la entrega. El alcance consta en revision_humana.md, también adjunto; no se confunde esa conformidad con las pruebas automáticas ni con la autorización previa de envío a OpenCode.

7. Uso de los archivos incluidos

Los anexos muestran íntegramente las fuentes requeridas. También se adjuntan sus bytes exactos dentro del PDF, junto con evidencias revisadas: un lector con panel de archivos adjuntos permite guardarlas sin reconstruir líneas envueltas visualmente. No hay JavaScript ni ejecución automática del PDF.

`tools/ps_common.ps1` debe permanecer en su carpeta tools junto a los scripts.

Para repetir sobre un laboratorio propio equivalente, configurar PGPASSFILE fuera de Git y proteger sus permisos. No copiar una contraseña a un comando. Reemplazar PgBin y WorkDir por rutas locales; preparar un directorio de trabajo privado existente y confirmar que no hay escritores. Los comandos ejecutados siguen este patrón, con el usuario técnico correspondiente:

```
$env:PGPASSFILE = 'C:\ruta_privada\pgpass.conf'
$pgBinU6 = 'C:\Program Files\PostgreSQL\17\bin'
$trabajoU6 = 'C:\ruta_privada\u6_runtime'
pwsh -NoProfile -File .\backup_foodstore.ps1 -DbUser u5_bootstrap `
  -PgBin $pgBinU6 -WorkDir $trabajoU6 -ConfirmNoWrites
pwsh -NoProfile -File .\restore_test.ps1 -DbUser u5_bootstrap `
  -PgBin $pgBinU6 -WorkDir $trabajoU6
```

Los valores predeterminados de este laboratorio son host 127.0.0.1 y puerto 55605; se pueden indicar explícitamente. Un destino existente se conserva: el script no lo elimina para repetir. La copia, claves y logs privados quedan fuera de los fuentes entregados. Los horarios recurrentes permanecen como plan.

Referencias primarias

- PostgreSQL 17: Respaldo y restauración.
- PostgreSQL 17: pg_dump y pg_restore.
- PostgreSQL 17: Monitoreo de actividad.
- PostgreSQL 17: Mantenimiento rutinario y vacuum/ANALYZE.
- OpenCode: Instrucciones AGENTS.md.

Anexos - Índice de fuentes

Los listados siguientes contienen las fuentes públicas completas. SPEC_U6.md ya está representada íntegramente en la primera página y no se duplica como listado. Todos los archivos, incluida SPEC, también se adjuntan individualmente dentro del PDF con sus bytes originales. Las evidencias y conversaciones de IA adicionales se adjuntan sin repetirlas como listados de código; el cuerpo contiene su explicación y los resultados pertinentes. Los ajustes visuales de líneas del listado no modifican esos archivos.

Anexo 1 - AGENTS.md 10

Anexo 2 - backup_foodstore.ps1 11

Anexo 3 - health_check.sql 13

Anexo 4 - maintenance.sql 14

Anexo 5 - restore_test.ps1 16

Anexo 6 - validaciones.sql 18

Anexo 7 - tools/ps_common.ps1 19

Integridad de los adjuntos

Archivo público	SHA-256 de los bytes originales
SPEC_U6.md	0c071839e8565c85b973cb027f7ac08aa493c912ecb7dfd04840a6ec621efaf8
AGENTS.md	9060f3ee7d56e02f754258d70a589a35e560c0054183160085e91fbeff5667a9
backup_foodstore.ps1	53612efb8f616d8cf1ff763d998d1155dce13d40b6aa1784e634d299404aa9f5
health_check.sql	948cbbb741032238ac6609e65338a6890bf25c0b41fd079a345943ed5f68fede
maintenance.sql	18f3becf82ebf07db6899692dbe7eb1c57838cf1528699a75ca7c0627bf0840b
restore_test.ps1	ab21a4341508d89aacc5fb5479d81f9042704ffb682d8705905ec58baeb7c8e9
validaciones.sql	8198129eaf53a1e96e432a02c0659f5c4eb7d094de2e05ad9deb022e1cb53849
tools/ps_common.ps1	bc5a4e87493312e4954d2e4849d756576fe6566c0256e902b4d770f1add81392
docs/revision_humana.md	ffef85b6b2ff28ac4e3b660822ebeaa7f6af2f9b87ba0cbfe9b2c5d14526982f
docs/ia/consentimiento_envio.md	fdffc2ebd4e7242ae8d7f30e277de33c7de4169d854aff76a919f02ca58a77c7
docs/ia/u6_generacion_adjuntos/01_SPEC_U6.md	0c071839e8565c85b973cb027f7ac08aa493c912ecb7dfd04840a6ec621efaf8
docs/ia/u6_generacion_adjuntos/02_AGENTS.md	9060f3ee7d56e02f754258d70a589a35e560c0054183160085e91fbeff5667a9
docs/ia/u6_generacion_metadatos.json	bb288cfc0c8e1be9d3ee0212137412d58b43616321d9c78966178f1667b0bbfc
docs/ia/u6_generacion_prompt.txt	6459e4e622fe3a3773031668aadfd9024bacee67d52591d38d85708614127e3c
docs/ia/u6_generacion_respuesta.md	b42ff831015a9042354240e9f5f4b567528389f9d3b40f19e11c403c4fc50752
docs/ia/u6_generacion_validacion.md	7b9611e6d8d5423fbed8792912f9335b21d995ac19f1b9c0be57afd43b80370b
docs/evidencias/backup_ejecucion.json	d48d68a11dec3b204ca98128a60d5987afe02a466e85b0d9af13e5679b76fc5e
docs/evidencias/cierre_conservacion.json	b0bda77927b8f72eebef22a92b54ec7389625a81fdc437ad70b6d92da2643f92
docs/evidencias/continuidad_u5_u6.json	1aec9b1dbc27024d8ae60dc584ab629fc31f2c2ed56d9e4a71a27a80d3cf073e
docs/evidencias/fallo_pg_dump_controlado.json	57fadb14bc30089d1d93c87d685a51afdafaceadb521d67144e9f0bc5320860a
docs/evidencias/health_check_ejecucion.json	4b6fb55c2da97963fd5aa9daf6ed5f0c3f210bd6abf5f5d336c354c530ca898f
docs/evidencias/health_check_salida.txt	da3c0bbe10d0f597b6fd55a93169e6ef25e148968b1811749d026bec8f5f6277
docs/evidencias/maintenance_ejecucion.json	9d47958415926cc4a4d0b2b37ace8a4a2286dae17410aec237b0772fcd4c94d8
docs/evidencias/maintenance_salida.txt	1af008def9414b5bb47bf0c1d6c3f4b58750b694c75876f8382b1060b4f102a6
docs/evidencias/mantenimiento_antes.json	5e7beda49527b87606aa9d265f6db2758a548aacd0cd599afa8461e78db63fa1
docs/evidencias/mantenimiento_despues.json	219aebbb520648ff5c21282c2af02e2281e97dab7fdf30aa60326bc4f4e7622f
docs/evidencias/respaldo_log.txt	c84f7bdcd0683618290c16f1e46b9ce758d4a2bdedfdd9521e8b48820845633e
docs/evidencias/respaldo_manifest.json	4e64fc07f117b7cd550fc6d703f27469902510123e0b60b27a8b9ead17dab1f8
docs/evidencias/restauracion_log.txt	e9566fd2e8d695d59edd1a92f832260a0ddaddacd4f81a73e14cf0f3c6da8395
docs/evidencias/restauracion_resultado.json	09d6a9f6e57e3ae8c25c5841d7d5fd9b19c0b945799e946c03088ecd23f6afc1
docs/evidencias/restore_ejecucion.json	a9e53087cbbb53e6230bdf0fa1f089f6233bca15eb981c39f59213b5fb8e3e9f
docs/evidencias/restore_existing_ejecucion.json	9a9b3647afccf0277aa6f1d254ac652a1a1e7aa5543c1237f943463d38da5c3e
docs/evidencias/validaciones_ejecucion.json	c5719f60fe5fea28698d8ee96e3a49b03813078e33874b7c2d083f6655588cef
docs/evidencias/validaciones_salida.txt	8f74bc65946733dd720e16b0758b315302d6cfc320c21f97dbff4815fc206dcd

Anexo 1 - AGENTS.md

Fuente íntegra | UTF-8 | SHA-256: 9060f3ee7d56e02f754258d70a589a35e560c0054183160085e91fbeff5667a9

```
# Instrucciones del laboratorio U6
- Para la consulta de generación de OpenCode: leer SPEC_U6.md antes de
  proponer scripts y responder sin herramientas ni ejecución. El operador
  revisará y probará con sus herramientas. No inventar métricas o resultados.
- PowerShell 7, PostgreSQL 17 nativo en Windows. Parámetros HostName, Port,
  DbUser, PgBin, WorkDir; nunca usar $Host/$HOME como variables propias.
- Conexiones con -w, PGPASSFILE externo y PGCONNECT_TIMEOUT. No contraseñas
  en scripts, prompts o logs. No cargar .env, perfiles ni datos personales.
- Base principal literal foodstore. Crear sólo foodstore_restore_test con
  createdb --template=template0. Si existe, abortar; nunca DROP/--clean/--create.
- Verificar inmediatamente LASTEXITCODE de cada ejecutable. psql -X con
  ON_ERROR_STOP=1. Error nativo o validación falsa => exit 1 y log FAILED.
- Backup -Fc con nombre UTC fechado único, archivo .partial hasta éxito;
  luego validar TOC con pg_restore --list, tamaño y SHA-256, mover sin sobrescribir
  a .dump y guardar manifiesto JSON. Logs y dump fuera del repositorio.
- Seleccionar último respaldo completado mediante manifiesto exitoso y
  comprobar tamaño/hash; no elegir un parcial por su fecha. No sobrescribir.
- Health: tamaño, numbackends/transacciones, pg_stat_user_tables y consultas
  activas. No mostrar datos de usuarios ni SQL con credenciales o literales.
- Maintenance: observar configuración autovacuum y estado antes de la acción;
  ANALYZE puntual sólo con decisión justificada. Sin VACUUM FULL ni cambios
  de autovacuum globales. Distinguir estimaciones de COUNT real.
- Modelo real: public.categoria(2), producto(50000), usuario(20001),
  pedido(225000), detalle_pedido(499997). usuario equivale a cliente; no crear
  otra tabla ni eliminar columnas. Son conteos de U5 a verificar, no resultados U6.
- detalle tiene PK(pedido_id,producto_id), cantidad y precio_unitario;
  subtotal GENERATED. pedido.total deriva de detalles. U4 agregó FNBC y MV
  diaria; TP5 tiene MV mensual; U5 agregó datos_anon y funciones DEFINER.
- Conteos e invalidaciones deben producir JSON de agregados únicamente;
  compararlos de forma ejecutable con línea base anterior al backup.
- RTO: Stopwatch justo antes de createdb, detener tras validación SQL y
  comparaciones exitosas. Preflight fuera del intervalo. RPO usa inicio del
  backup y principio del ensayo, nunca la hora de cierre como si fuera snapshot.
- pg_restore --no-owner --no-privileges --exit-on-error -d destino. No afirmar
  que recupera roles/ACL: los DEFINER cambian de dueño; el restaurador debe
  aislar la base y revocar conexión/esquemas/EXECUTE públicos sin reabrirla
  a app_web. No ejecutar roles.sql U5 sobre estos nombres distintos.
- Entrega: PDF único con SPEC e informe breve separados de los anexos.
  No programar tareas reales ni enviar a terceros.
```

Anexo 2 - backup_foodstore.ps1

Fuente íntegra | UTF-8 | SHA-256: 53612efb8f616d8cf1ff763d998d1155dce13d40b6aa1784e634d299404aa9f5

```
#requires -Version 7.5
# Base: OpenCode real u6_generacion; integracion/revision Codex documentada en docs/ia/.
[CmdletBinding()]
param(
    [string]$HostName = '127.0.0.1',
    [int]$Port = 55605,
    [Parameter(Mandatory)][string]$DbUser,
    [Parameter(Mandatory)][string]$PgBin,
    [Parameter(Mandatory)][string]$WorkDir,
    [switch]$ConfirmNoWrites
)
$ErrorActionPreference = 'Stop'
$PSNativeCommandUseErrorActionPreference = $false
$env:PGCONNECT_TIMEOUT = '10'
$env:PGAPPNAME = 'u6_backup'
$log = $null; $phase = 'preflight'
. (Join-Path $PSScriptRoot 'tools/ps_common.ps1')
try {
    if ($HostName -ne '127.0.0.1' -or ($Port -lt 1 -or $Port -gt 65535) -or
        -not $ConfirmNoWrites) { throw 'Endpoint o ventana sin escrituras no confirmados.' }
    $WorkDir = (Get-Item -LiteralPath $WorkDir).FullName
    if (-not (Test-Path -LiteralPath $WorkDir -PathType Container)) {
        throw 'WorkDir no es directorio.'
    }
    Assert-Private $WorkDir
    $BackupDir=Join-Path $WorkDir 'backups'; $LogDir=Join-Path $WorkDir 'logs'
    foreach ($directory in @($BackupDir,$LogDir)) {
        [IO.Directory]::CreateDirectory($directory) | Out-Null
        Assert-Private $directory
    }
    $token = New-RunToken
    $stem = "foodstore_$token"
    $log = New-Writer (Join-Path $LogDir "$stem.log"); $log.AutoFlush = $true
    Log 'START backup'
    if (-not $env:PGPASSFILE -or
        -not (Test-Path -LiteralPath $env:PGPASSFILE -PathType Leaf)) { throw 'Falta PGPASSFILE.' }
    Assert-Private $env:PGPASSFILE
    Assert-Clients @('psql','pg_dump','pg_restore')
    $sql = Join-Path $PSScriptRoot 'validaciones.sql'
    if (-not (Test-Path -LiteralPath $sql -PathType Leaf)) { throw 'Falta validaciones.sql.' }
    $conn = @('-h', $HostName, '-p', "$Port", '-U', $DbUser, '-w')
    $psql = @('-X', '-q', '-A', '-t', '-v', 'ON_ERROR_STOP=1') + $conn + @('-d', 'foodstore')
    $check = @'
SELECT (current_setting('server_version_num')::int BETWEEN 170000 AND 179999
 AND (SELECT rolsuper FROM pg_roles WHERE rolname=current_user))::int;
'@
    if ((Pg 'psql' ($psql + @('-c',$check)) | Out-String).Trim() -ne '1') {
        throw 'Se requiere servidor 17 y administrador bootstrap.'
    }
    $phase = 'baseline anterior'
    $baseline = (Pg 'psql' ($psql + @('-f', $sql)) | Out-String) | ConvertFrom-Json -AsHashtable
    Assert-Baseline $baseline
    $partial = Join-Path $BackupDir "$stem.partial"; $dump = Join-Path $BackupDir "$stem.dump"
    if ((Test-Path -LiteralPath $partial) -or (Test-Path -LiteralPath $dump)) { throw 'Colision de nombre
  s.' }
    $phase = 'pg_dump'
    $started = [DateTimeOffset]::UtcNow
    Pg 'pg_dump' ($conn + @('-d', 'foodstore', '-Fc', '-f', $partial)) | Out-Null
    if ((Get-Item -LiteralPath $partial).Length -le 0) { throw 'Respaldo vacio.' }
    $phase = 'TOC y segunda baseline'
    Pg 'pg_restore' @('--list', $partial) | Out-Null
    $after = (Pg 'psql' ($psql + @('-f', $sql)) | Out-String) | ConvertFrom-Json -AsHashtable
    Assert-Baseline $after
    foreach ($section in @('counts','fingerprints','invalid')) {
        foreach ($key in $baseline[$section].Keys) {
            if ($baseline[$section][$key] -cne $after[$section][$key]) { throw 'Cambio observable durante
   el respaldo.' }
        }
    }
    $size = (Get-Item -LiteralPath $partial).Length
    $hash = (Get-FileHash -LiteralPath $partial -Algorithm SHA256).Hash.ToLowerInvariant()
    $phase = 'publicacion'
```

```
    [IO.File]::Move($partial, $dump)
    $ended = [DateTimeOffset]::UtcNow
    $manifest = [ordered]@{
        status='SUCCESS'; database='foodstore'; file="$stem.dump"; size=$size; sha256=$hash
        startUtc=$started.ToString('o'); endUtc=$ended.ToString('o'); baseline=$baseline
        startUtcMinus03=$started.ToOffset([TimeSpan]::FromHours(-3)).ToString('o')
        endUtcMinus03=$ended.ToOffset([TimeSpan]::FromHours(-3)).ToString('o')
    }
    $manifestPartial = Join-Path $LogDir "$stem.manifest.partial"
    $writer = New-Writer $manifestPartial
    try { $writer.Write(($manifest | ConvertTo-Json -Depth 8)) } finally { $writer.Dispose() }
    [IO.File]::Move($manifestPartial, (Join-Path $LogDir "$stem.manifest.json"))
    Log "SUCCESS backup archivo=$stem.dump bytes=$size sha256=$hash"
} catch {
    Log "FAILED backup fase=$phase"
    exit 1
} finally {
    if ($null -ne $log) { $log.Dispose() }
}
exit 0
```

Anexo 3 - health_check.sql

Fuente íntegra | UTF-8 | SHA-256: 948cbbb741032238ac6609e65338a6890bf25c0b41fd079a345943ed5f68fede

```
-- Unidad 6 - Food Store. Exequiel Arancibia / Joaquin Romero.
-- Implementacion tecnica: Codex, segun SPEC_U6.md y AGENTS.md.
-- Uso: psql -X -w -v ON_ERROR_STOP=1 -d foodstore -f health_check.sql
-- Solo agregados/estadisticas. No exporta filas ni texto SQL de sesiones.
\set ON_ERROR_STOP on
\set QUIET 1
\pset pager off
\pset null '(NULL)'
BEGIN READ ONLY;
SET LOCAL search_path = pg_catalog;
SET LOCAL TIME ZONE 'America/Argentina/Buenos_Aires';
SET LOCAL application_name = 'u6_health_check';
DO $base_correcta$
BEGIN
    IF pg_catalog.current_database() <> 'foodstore' THEN
        RAISE EXCEPTION USING ERRCODE = '22023',
            MESSAGE = 'health_check.sql requiere la base foodstore';
    END IF;
END
$base_correcta$;
\echo 'CONTROL 1 - Tamano de la base principal'
SELECT pg_catalog.current_database() AS base,
       pg_catalog.clock_timestamp() AS observado_en,
       pg_catalog.pg_database_size(pg_catalog.current_database()) AS bytes,
       pg_catalog.pg_size_pretty(pg_catalog.pg_database_size(pg_catalog.current_database())) AS tamano;
\echo 'CONTROL 2 - Conexiones y contadores acumulados; no son tasas ni filas de negocio'
SELECT datname AS base, numbackends AS conexiones,
       xact_commit AS transacciones_confirmadas,
       xact_rollback AS transacciones_revertidas,
       deadlocks AS interbloqueos, stats_reset AS reinicio_contadores
FROM pg_catalog.pg_stat_database
WHERE datname = pg_catalog.current_database();
\echo 'CONTROL 3 - Estado estimado de tablas; n_live_tup NO equivale a COUNT(*)'
SELECT schemaname AS esquema, relname AS tabla,
       n_live_tup AS filas_vivas_estimadas,
       n_dead_tup AS tuplas_muertas_estimadas,
       n_mod_since_analyze AS cambios_estimados_desde_analyze,
       last_vacuum, last_autovacuum, last_analyze, last_autoanalyze
FROM pg_catalog.pg_stat_user_tables
ORDER BY n_dead_tup DESC, schemaname, relname;
\echo 'CONTROL 4 - Consultas activas ajenas al chequeo; sin texto SQL, usuario ni direccion'
-- pg_stat_activity.query solo se usa para una clasificacion de lista cerrada.
-- WITH puede contener escrituras: no se rotula automaticamente como lectura.
-- Los application_name no reconocidos tambien se ocultan porque son texto libre.
SELECT pid, backend_type AS tipo_backend, state AS estado,
       pg_catalog.clock_timestamp() - query_start AS duracion,
       wait_event_type AS tipo_espera, wait_event AS espera,
       CASE
           WHEN application_name IN ('psql','pg_dump','pg_restore','u6_backup','u6_restore_test')
               THEN application_name
           ELSE '[aplicacion omitida]'
       END AS aplicacion,
       CASE
           WHEN query ~* '^[[:space:]]*SELECT([[:space:](]|$)' THEN 'SELECT'
           WHEN query ~* '^[[:space:]]*WITH([[:space:](]|$)' THEN 'WITH'
           WHEN query ~* '^[[:space:]]*INSERT([[:space:]]|$)' THEN 'INSERT'
           WHEN query ~* '^[[:space:]]*UPDATE([[:space:]]|$)' THEN 'UPDATE'
           WHEN query ~* '^[[:space:]]*DELETE([[:space:]]|$)' THEN 'DELETE'
           WHEN query ~* '^[[:space:]]*ANALYZE([[:space:](]|$)' THEN 'ANALYZE'
           WHEN query ~* '^[[:space:]]*VACUUM([[:space:](]|$)' THEN 'VACUUM'
           ELSE 'OTRA / NO VISIBLE'
       END AS clase_de_consulta
FROM pg_catalog.pg_stat_activity
WHERE datname = pg_catalog.current_database()
  AND state = 'active'
  AND pid <> pg_catalog.pg_backend_pid()
ORDER BY query_start NULLS LAST, pid;
COMMIT;
-- Cero consultas activas es un resultado valido; no se fabrica actividad.
-- Usuarios sin pg_read_all_stats pueden tener visibilidad limitada de otras
-- sesiones. Las estadisticas pueden retrasarse; NULL no prueba daemon apagado.
```

Anexo 4 - maintenance.sql

Fuente íntegra | UTF-8 | SHA-256: 18f3becf82ebf07db6899692dbe7eb1c57838cf1528699a75ca7c0627bf0840b

```
-- Unidad 6 - Food Store. Exequiel Arancibia / Joaquin Romero.
-- Implementacion tecnica: Codex, segun SPEC_U6.md y observacion previa.
-- Uso: psql -X -w -v ON_ERROR_STOP=1 -d foodstore -f maintenance.sql
-- No modifica filas, permisos ni autovacuum. Actualiza estadisticas puntuales.
-- Ejecutar como operador tecnico autorizado, no como login de aplicacion.
\set ON_ERROR_STOP on
\set QUIET 1
\pset pager off
\pset null '(NULL)'
SET search_path = pg_catalog;
SET TIME ZONE 'America/Argentina/Buenos_Aires';
SET application_name = 'u6_maintenance';
SET lock_timeout = '5s';
SET statement_timeout = '60s';
DO $base_correcta$
BEGIN
    IF pg_catalog.current_database() <> 'foodstore' THEN
        RAISE EXCEPTION USING ERRCODE = '22023',
            MESSAGE = 'maintenance.sql requiere la base foodstore';
    END IF;
END
$base_correcta$;
\echo 'ANTES 1 - Configuracion efectiva de autovacuum y recoleccion de estadisticas'
SELECT pg_catalog.clock_timestamp() AS observado_en,
       pg_catalog.current_setting('autovacuum') AS autovacuum,
       pg_catalog.current_setting('track_counts') AS track_counts,
       pg_catalog.current_setting('autovacuum_analyze_threshold') AS umbral_analyze,
       pg_catalog.current_setting('autovacuum_analyze_scale_factor') AS factor_analyze,
       pg_catalog.current_setting('autovacuum_vacuum_threshold') AS umbral_vacuum,
       pg_catalog.current_setting('autovacuum_vacuum_scale_factor') AS factor_vacuum;
\echo 'ANTES 2 - Tablas de la copia: estimaciones e historiales, con opciones por tabla'
SELECT s.schemaname AS esquema, s.relname AS tabla,
       s.n_live_tup AS filas_vivas_estimadas,
       s.n_dead_tup AS tuplas_muertas_estimadas,
       s.n_mod_since_analyze AS cambios_estimados_desde_analyze,
       s.last_vacuum, s.last_autovacuum, s.last_analyze, s.last_autoanalyze,
       c.reloptions AS opciones_de_tabla
FROM pg_catalog.pg_stat_user_tables AS s
JOIN pg_catalog.pg_class AS c ON c.oid = s.relid
ORDER BY s.n_dead_tup DESC, s.schemaname, s.relname;
\echo 'ANTES 3 - Conteos reales de las dos tablas elegidas; no inferir vaciado de una estimacion 0'
SELECT 'pedido' AS tabla, pg_catalog.count(*) AS filas_reales FROM public.pedido
UNION ALL
SELECT 'detalle_pedido', pg_catalog.count(*) FROM public.detalle_pedido;
\echo 'DECISION - ANALYZE puntual si last_analyze y last_autoanalyze son ambos NULL por tabla'
-- Observacion previa de esta copia: autovacuum=on, ambas tablas tienen
-- filas reales y estimaciones/historiales recien iniciados. Ausencia de
-- historial NO demuestra ausencia de pg_statistic. Se admite refrescar una
-- linea base documentada de la copia, sin reclamar tuplas muertas inexistentes.
-- No hay motivo observado para VACUUM FULL ni para desactivar autovacuum.
SELECT relname AS tabla,
       CASE WHEN last_analyze IS NULL AND last_autoanalyze IS NULL
            THEN 'ANALYZE puntual: establecer linea base de la copia'
            ELSE 'Sin ANALYZE manual: ya hay historial registrado'
       END AS decision
FROM pg_catalog.pg_stat_user_tables
WHERE relid IN ('public.pedido'::pg_catalog.regclass,
                'public.detalle_pedido'::pg_catalog.regclass)
ORDER BY relname;
-- Cada sentencia de psql se confirma por separado. ANALYZE es admisible
-- dentro de DO; no se utiliza VACUUM, que tiene otras restricciones.
DO $analyze_condicional$
BEGIN
    IF EXISTS (
        SELECT 1 FROM pg_catalog.pg_stat_user_tables
        WHERE relid = 'public.pedido'::pg_catalog.regclass
          AND last_analyze IS NULL AND last_autoanalyze IS NULL
    ) THEN
        EXECUTE 'ANALYZE public.pedido';
        RAISE NOTICE 'ANALYZE public.pedido completado para la linea base de la copia';
    END IF;
    IF EXISTS (
```

```
        SELECT 1 FROM pg_catalog.pg_stat_user_tables
        WHERE relid = 'public.detalle_pedido'::pg_catalog.regclass
          AND last_analyze IS NULL AND last_autoanalyze IS NULL
    ) THEN
        EXECUTE 'ANALYZE public.detalle_pedido';
        RAISE NOTICE 'ANALYZE public.detalle_pedido completado para la linea base de la copia';
    END IF;
END
$analyze_condicional$;
-- Descartar cualquier cache estadistica del lector antes de la observacion
-- posterior; el servidor puede tener un breve retraso de publicacion.
DO $lectura_nueva$
BEGIN
    PERFORM pg_catalog.pg_stat_clear_snapshot();
END
$lectura_nueva$;
\echo 'DESPUES - Valores realmente visibles, sin sustituir NULL por un resultado inventado'
SELECT schemaname AS esquema, relname AS tabla,
       n_live_tup AS filas_vivas_estimadas,
       n_dead_tup AS tuplas_muertas_estimadas,
       n_mod_since_analyze AS cambios_estimados_desde_analyze,
       last_analyze, last_autoanalyze,
       analyze_count AS analyze_manuales_registrados
FROM pg_catalog.pg_stat_user_tables
WHERE relid IN ('public.pedido'::pg_catalog.regclass,
                'public.detalle_pedido'::pg_catalog.regclass)
ORDER BY relname;
-- Si aparece historial al repetir, la condicion evita ANALYZE innecesario.
-- Esta politica es del laboratorio; NULL aislado no prueba estadisticas malas.
```

Anexo 5 - restore_test.ps1

Fuente íntegra | UTF-8 | SHA-256: ab21a4341508d89aacc5fb5479d81f9042704ffb682d8705905ec58baeb7c8e9

```
#requires -Version 7.5
# Base: OpenCode real u6_generacion; integracion/revision Codex documentada en docs/ia/.
[CmdletBinding()]
param(
    [string]$HostName = '127.0.0.1',
    [int]$Port = 55605,
    [Parameter(Mandatory)][string]$DbUser,
    [Parameter(Mandatory)][string]$PgBin,
    [Parameter(Mandatory)][string]$WorkDir,
    [string[]]$BusinessRoles = @('app_web','admin_datos','rol_app_lectura',
        'rol_app_escritura','rol_soporte','rol_reportes')
)
$ErrorActionPreference = 'Stop'
$PSNativeCommandUseErrorActionPreference = $false
$env:PGCONNECT_TIMEOUT = '10'
$env:PGAPPNAME = 'u6_restore_test'
$log = $null; $timer = $null; $phase = 'preflight'
. (Join-Path $PSScriptRoot 'tools/ps_common.ps1')
try {
    if ($HostName -ne '127.0.0.1' -or ($Port -lt 1 -or $Port -gt 65535)) {
        throw 'Endpoint no permitido.'
    }
    $WorkDir = (Get-Item -LiteralPath $WorkDir).FullName
    if (-not (Test-Path -LiteralPath $WorkDir -PathType Container)) {
        throw 'WorkDir no es directorio.'
    }
    Assert-Private $WorkDir
    $BackupDir=Join-Path $WorkDir 'backups'; $LogDir=Join-Path $WorkDir 'logs'
    foreach ($directory in @($BackupDir,$LogDir)) { Assert-Private $directory }
    $token = New-RunToken
    $log = New-Writer (Join-Path $LogDir "restore_$token.log"); $log.AutoFlush = $true
    Log 'START restore_test'
    if (-not $env:PGPASSFILE -or
        -not (Test-Path -LiteralPath $env:PGPASSFILE -PathType Leaf)) { throw 'Falta PGPASSFILE.' }
    Assert-Private $env:PGPASSFILE
    Assert-Clients @('psql','createdb','pg_restore')
    $expected = @('app_web','admin_datos','rol_app_lectura',
        'rol_app_escritura','rol_soporte','rol_reportes')
    if (@(Compare-Object ($BusinessRoles | Sort-Object) ($expected | Sort-Object)).Count) {
        throw 'Se requieren los seis roles reales del laboratorio.'
    }
    $roles = ($BusinessRoles | ForEach-Object { '"' + $_.Replace('"','""') + '"' }) -join ', '
    $literals = ($BusinessRoles | ForEach-Object { "'" + $_.Replace("'","''") + "'" }) -join ', '
    $rolesLiteral = "'" + $roles.Replace("'","''") + "'"
    $conn = @('-h', $HostName, '-p', "$Port", '-U', $DbUser, '-w')
    $psql = @('-X', '-q', '-A', '-t', '-v', 'ON_ERROR_STOP=1') + $conn
    $checkSql = @"
SELECT json_build_object('version',current_setting('server_version_num')::int,
 'admin',(SELECT rolsuper FROM pg_roles WHERE rolname=current_user),
 'exists',EXISTS(SELECT FROM pg_database WHERE datname='foodstore_restore_test'),
 'roles',(SELECT count(*) FROM pg_roles WHERE rolname IN ($literals)
          AND NOT rolsuper AND rolname<>current_user));
"@
    $check = (Pg 'psql' ($psql + @('-d','postgres','-c',$checkSql)) | Out-String) |
        ConvertFrom-Json -AsHashtable
    if ($check.version -lt 170000 -or $check.version -ge 180000 -or $check.admin -ne $true -or
        $check.exists -ne $false -or $check.roles -ne 6) {
        throw 'Servidor, administrador, destino o roles no validos.'
    }
    $sql = Join-Path $PSScriptRoot 'validaciones.sql'
    if (-not (Test-Path -LiteralPath $sql -PathType Leaf)) { throw 'Falta validaciones.sql.' }
    $chosen = Read-BackupManifest
    $m=$chosen.Manifest; $start=$chosen.Start; $dump=$chosen.Dump
    $dest = $psql + @('-d','foodstore_restore_test')
    $isolation = @'
DO $isolation$
DECLARE n record;
BEGIN
  FOR n IN SELECT nspname FROM pg_namespace
           WHERE nspname !~ '^pg_' AND nspname <> 'information_schema'
  LOOP
    EXECUTE format('REVOKE USAGE, CREATE ON SCHEMA %I FROM PUBLIC, %s', n.nspname, __ROLES__);
```

```
    EXECUTE format('REVOKE EXECUTE ON ALL ROUTINES IN SCHEMA %I FROM PUBLIC, %s', n.nspname, __ROLES__);
  END LOOP;
END
$isolation$;
'@
    $isolation = $isolation.Replace('__ROLES__', $rolesLiteral)
    # Solo operadores en loopback: createdb y el cierre son comandos consecutivos, no atomicos.
    $phase = 'creacion y aislamiento'
    $trialStart = [DateTimeOffset]::UtcNow
    $age = $trialStart - $start
    $timer = [Diagnostics.Stopwatch]::StartNew()
    Pg 'createdb' ($conn + @('--maintenance-db=postgres','--template=template0','foodstore_restore_test')
  ) | Out-Null
    $fence = "ALTER DATABASE foodstore_restore_test CONNECTION LIMIT 0; " +
        "REVOKE CONNECT, TEMPORARY ON DATABASE foodstore_restore_test FROM PUBLIC, $roles;"
    Pg 'psql' ($psql + @('-d','postgres','-c',$fence)) | Out-Null
    $active = "SELECT count(*) FROM pg_stat_activity WHERE datname='foodstore_restore_test';"
    if ((Pg 'psql' ($psql + @('-d','postgres','-c',$active)) | Out-String).Trim() -ne '0') {
        throw 'Hay otras conexiones al destino; no se restaura.'
    }
    $phase = 'pg_restore'
    Pg 'pg_restore' ($conn + @('--no-owner','--no-privileges','--exit-on-error','-d','foodstore_restore_t
  est',$dump)) | Out-Null
    $phase = 'aislamiento final y validacion'
    Pg 'psql' ($dest + @('-c', $isolation)) | Out-Null
    Assert-Isolation $dest $literals
    $actual = (Pg 'psql' ($dest + @('-f', $sql)) | Out-String) | ConvertFrom-Json -AsHashtable
    Assert-Baseline $actual
    foreach ($section in @('counts','fingerprints','invalid')) {
        foreach ($key in $m.baseline[$section].Keys) {
            if ($m.baseline[$section][$key] -cne $actual[$section][$key]) { throw 'Comparacion falsa.' }
        }
    }
    $timer.Stop()
    $rto = $timer.Elapsed.TotalSeconds.ToString('F3',[Globalization.CultureInfo]::InvariantCulture)
    $rpo = $age.TotalSeconds.ToString('F3',[Globalization.CultureInfo]::InvariantCulture)
    Log ("medicion inicioUTC=$($trialStart.ToString('o')) RTO_s=$rto RPO_referencia_s=$rpo " +
        "inicioUTC03=$($trialStart.ToOffset([TimeSpan]::FromHours(-3)).ToString('o'))")
    Log "agregados=$($actual | ConvertTo-Json -Compress -Depth 6)"
    if ($timer.Elapsed.TotalSeconds -gt 120 -or $age.TotalHours -gt 24 -or
        $age.TotalSeconds -lt 0) { throw 'Objetivo RTO/RPO incumplido.' }
    $result = [ordered]@{
        status='SUCCESS'; database='foodstore_restore_test'; file=$m.file
        backupStartUtc=$m.startUtc; trialStartUtc=$trialStart.ToString('o')
        validatedUtc=[DateTimeOffset]::UtcNow.ToString('o'); baseline=$m.baseline; actual=$actual
        rtoSeconds=$timer.Elapsed.TotalSeconds; rpoReferenceSeconds=$age.TotalSeconds
        isolated=$true; roleAclRecovery=$false
    }
    $writer = New-Writer (Join-Path $LogDir "restore_$token.result.json")
    try { $writer.Write(($result | ConvertTo-Json -Depth 8)) } finally { $writer.Dispose() }
    Log "SUCCESS restore_test respaldo=$($m.file) destino_aislado=foodstore_restore_test"
} catch {
    if ($null -ne $timer -and $timer.IsRunning) { $timer.Stop() }
    Log "FAILED restore_test fase=$phase"
    exit 1
} finally {
    if ($null -ne $log) { $log.Dispose() }
}
exit 0
```

Anexo 6 - validaciones.sql

Fuente íntegra | UTF-8 | SHA-256: 8198129eaf53a1e96e432a02c0659f5c4eb7d094de2e05ad9deb022e1cb53849

```
-- Unidad 6 - Food Store. Exequiel Arancibia / Joaquin Romero.
-- Implementacion tecnica: Codex. Ejecutar con psql -X -w -A -t
-- -v ON_ERROR_STOP=1 -d foodstore[..._restore_test] -f validaciones.sql.
-- stdout: UN objeto JSON con counts, fingerprints, invalid. Sin filas ni PII.
-- La guia llama cliente a la entidad que aqui conserva public.usuario.
\set ON_ERROR_STOP on
\set QUIET 1
\pset pager off
BEGIN ISOLATION LEVEL REPEATABLE READ READ ONLY;
SET LOCAL search_path = pg_catalog;
SET LOCAL TIME ZONE 'America/Argentina/Buenos_Aires';
SET LOCAL DateStyle = 'ISO, YMD';
SET LOCAL application_name = 'u6_validaciones';
DO $base_correcta$
BEGIN
    IF pg_catalog.current_database() NOT IN ('foodstore','foodstore_restore_test') THEN
        RAISE EXCEPTION USING ERRCODE = '22023',
            MESSAGE = 'validaciones.sql requiere foodstore o foodstore_restore_test';
    END IF;
END
$base_correcta$;
-- Una instantanea por ejecucion evita mezclar conteos de varios instantes.
-- El orquestador debe evitar escritores entre baseline y snapshot del dump,
-- o compartir expresamente un snapshot exportado. Este archivo no lo presume.
-- Las huellas siguen U5: MD5 de la concatenacion ordenada de MD5 de cada fila
-- completa en JSON. Son checksums de conservacion, no anonimato ni firma.
-- Se fija zona/DateStyle para que timestamps tengan la misma representacion.
WITH categorias AS (
    SELECT pg_catalog.count(*) AS n,
           pg_catalog.md5(COALESCE(pg_catalog.string_agg(
               pg_catalog.md5(pg_catalog.row_to_json(t)::text),'' ORDER BY t.id),'')) AS huella
    FROM public.categoria AS t
), productos AS (
    SELECT pg_catalog.count(*) AS n,
           pg_catalog.count(*) FILTER (WHERE t.precio < 0 OR t.stock < 0) AS invalidos,
           pg_catalog.md5(COALESCE(pg_catalog.string_agg(
               pg_catalog.md5(pg_catalog.row_to_json(t)::text),'' ORDER BY t.id),'')) AS huella
    FROM public.producto AS t
), usuarios AS (
    SELECT pg_catalog.count(*) AS n,
           pg_catalog.md5(COALESCE(pg_catalog.string_agg(
               pg_catalog.md5(pg_catalog.row_to_json(t)::text),'' ORDER BY t.id),'')) AS huella
    FROM public.usuario AS t
), pedidos AS (
    SELECT pg_catalog.count(*) AS n,
           pg_catalog.md5(COALESCE(pg_catalog.string_agg(
               pg_catalog.md5(pg_catalog.row_to_json(t)::text),'' ORDER BY t.id),'')) AS huella
    FROM public.pedido AS t
), detalles AS (
    SELECT pg_catalog.count(*) AS n,
           pg_catalog.count(*) FILTER (WHERE t.cantidad <= 0 OR t.precio_unitario < 0) AS invalidos,
           pg_catalog.md5(COALESCE(pg_catalog.string_agg(
               pg_catalog.md5(pg_catalog.row_to_json(t)::text),''
               ORDER BY t.pedido_id,t.producto_id),'')) AS huella
    FROM public.detalle_pedido AS t
)
SELECT pg_catalog.jsonb_build_object(
    'counts', pg_catalog.jsonb_build_object(
        'categoria',c.n,'producto',pr.n,'usuario',u.n,'pedido',pe.n,'detalle_pedido',d.n),
    'fingerprints', pg_catalog.jsonb_build_object(
        'categoria',c.huella,'producto',pr.huella,'usuario',u.huella,
        'pedido',pe.huella,'detalle_pedido',d.huella),
    'invalid', pg_catalog.jsonb_build_object('productos',pr.invalidos,'detalles',d.invalidos)
) AS validacion
FROM categorias AS c CROSS JOIN productos AS pr CROSS JOIN usuarios AS u
CROSS JOIN pedidos AS pe CROSS JOIN detalles AS d;
COMMIT;
-- No agregar metadata variable a este JSON: los scripts registran fecha/base
-- y comparan los cinco counts/fingerprints y los dos ceros de forma explicita.
-- precio, stock, cantidad y precio_unitario son NOT NULL en el modelo real;
-- las condiciones invalid conservan exactamente los controles de la guia.
```

Anexo 7 - tools/ps_common.ps1

Fuente íntegra | UTF-8 | SHA-256: bc5a4e87493312e4954d2e4849d756576fe6566c0256e902b4d770f1add81392

```
# Funciones comunes factorizadas de la propuesta real OpenCode u6_generacion.
# Integracion/revision automatica Codex; no equivalen a prueba de ejecucion.
foreach ($key in @('PGHOSTADDR','PGSERVICE','PGSERVICEFILE','PGOPTIONS','PGPASSWORD')) {
    Remove-Item -LiteralPath "Env:$key" -ErrorAction SilentlyContinue
}
$env:PGCLIENTENCODING = 'UTF8'
function Log([string]$Text) {
    $line = "$([DateTimeOffset]::UtcNow.ToString('o')) $Text"
    if ($null -ne $log) { $log.WriteLine($line) }
    [Console]::Error.WriteLine($line)
}
function New-Writer([string]$Path) {
    $stream = [IO.File]::Open($Path, 'CreateNew', 'Write', 'None')
    [IO.StreamWriter]::new($stream, [Text.UTF8Encoding]::new($false))
}
function Assert-Private([string]$Path) {
    $item = Get-Item -LiteralPath $Path
    if ($item.Attributes -band [IO.FileAttributes]::ReparsePoint) { throw 'Ruta redirigida.' }
    $allowed = @([Security.Principal.WindowsIdentity]::GetCurrent().User.Value, 'S-1-5-18', 'S-1-5-32-544
  ')
    foreach ($ace in (Get-Acl -LiteralPath $Path).Access) {
        $sid = $ace.IdentityReference.Translate([Security.Principal.SecurityIdentifier]).Value
        if ($ace.AccessControlType -eq 'Allow' -and $sid -notin $allowed) { throw 'ACL no privada.' }
    }
    $parent = if ($item.PSIsContainer) { $item } else { $item.Directory }
    for ($d=$parent; $null -ne $d; $d=$d.Parent) {
        if (($d.Attributes -band [IO.FileAttributes]::ReparsePoint) -or
            (Test-Path -LiteralPath (Join-Path $d.FullName '.git'))) {
            throw 'Material privado dentro de Git o ruta redirigida.'
        }
    }
}
function Pg([string]$Name, [string[]]$Arguments) {
    $output = & (Join-Path $PgBin "$Name.exe") @Arguments 2>$null
    $code = $LASTEXITCODE
    if ($code -ne 0) { Log "FAILED native=$Name code=$code"; throw "${Name}: codigo $code" }
    $output
}
function Assert-Baseline($Value) {
    $tables = @('categoria','producto','usuario','pedido','detalle_pedido')
    $sets = @{ counts=$tables; fingerprints=$tables; invalid=@('productos','detalles') }
    if ($Value -isnot [Collections.IDictionary] -or $Value.Count -ne 3) { throw 'JSON base invalido.' }
    foreach ($section in $sets.Keys) {
        $part = $Value[$section]
        if ($part -isnot [Collections.IDictionary] -or $part.Count -ne $sets[$section].Count) {
            throw 'Claves invalidas.'
        }
        foreach ($key in $sets[$section]) {
            if (-not $part.Contains($key)) { throw 'Falta clave.' }
            $v = $part[$key]
            if ($section -eq 'fingerprints') {
                if ($v -isnot [string] -or $v -cnotmatch '^[0-9a-f]{32}$') { throw 'Huella invalida.' }
            } elseif (($v -isnot [long] -and $v -isnot [int]) -or $v -lt 0 -or
                      ($section -eq 'invalid' -and $v -ne 0)) { throw 'Conteo o integridad invalidos.' }
        }
    }
}
function Assert-Isolation([string[]]$PsqlArguments, [string]$RoleLiterals) {
    $check = @'
SELECT (d.datconnlimit=0 AND NOT EXISTS (
 SELECT 1 FROM pg_catalog.pg_roles r WHERE r.rolname IN (__LITERALS__)
 AND (pg_catalog.has_database_privilege(r.rolname,d.datname,'CONNECT')
   OR pg_catalog.has_database_privilege(r.rolname,d.datname,'TEMPORARY'))
) AND NOT EXISTS (
 SELECT 1 FROM pg_catalog.pg_roles r CROSS JOIN pg_catalog.pg_namespace n
 WHERE r.rolname IN (__LITERALS__) AND n.nspname !~ '^pg_'
 AND n.nspname<>'information_schema'
 AND (pg_catalog.has_schema_privilege(r.rolname,n.oid,'USAGE')
   OR pg_catalog.has_schema_privilege(r.rolname,n.oid,'CREATE'))
) AND NOT EXISTS (
 SELECT 1 FROM pg_catalog.pg_roles r CROSS JOIN pg_catalog.pg_proc p
 JOIN pg_catalog.pg_namespace n ON n.oid=p.pronamespace
```

```
 WHERE r.rolname IN (__LITERALS__) AND n.nspname !~ '^pg_'
 AND n.nspname<>'information_schema'
 AND pg_catalog.has_function_privilege(r.rolname,p.oid,'EXECUTE')
))::int FROM pg_catalog.pg_database d WHERE d.datname=current_database();
'@
    $check = $check.Replace('__LITERALS__',$RoleLiterals)
    if ((Pg 'psql' ($PsqlArguments + @('-c',$check)) | Out-String).Trim() -ne '1') {
        throw 'Aislamiento final no verificado.'
    }
}
function New-RunToken {
    $stamp = [DateTimeOffset]::UtcNow.ToString('yyyyMMddTHHmmssfffffffZ',
        [Globalization.CultureInfo]::InvariantCulture)
    "$stamp`_$([guid]::NewGuid().ToString('N'))"
}
function Assert-Clients([string[]]$Names) {
    foreach ($name in $Names) {
        $version = Pg $name @('--version') | Out-String
        if ($version -notmatch '\(PostgreSQL\) 17(?:\.|\s|$)') { throw 'Se requieren clientes 17.' }
    }
}
function Read-BackupManifest {
    $candidates = @(foreach ($file in Get-ChildItem -LiteralPath $LogDir -Filter '*.manifest.json' -File)
   {
        Assert-Private $file.FullName
        $m = Get-Content -LiteralPath $file.FullName -Raw |
            ConvertFrom-Json -AsHashtable -DateKind String
        if ($m.status -ceq 'SUCCESS') {
            $end = [DateTimeOffset]::ParseExact($m.endUtc,'o',[Globalization.CultureInfo]::InvariantCultu
  re)
            [pscustomobject]@{ Manifest=$m; End=$end }
        }
    })
    if ($candidates.Count -eq 0) { throw 'No hay manifiesto exitoso.' }
    $chosen = $candidates | Sort-Object End -Descending | Select-Object -First 1
    $m = $chosen.Manifest
    $start = [DateTimeOffset]::ParseExact($m.startUtc,'o',[Globalization.CultureInfo]::InvariantCulture)
    if ($m.database -cne 'foodstore' -or $start.Offset -ne [TimeSpan]::Zero -or
        $chosen.End.Offset -ne [TimeSpan]::Zero -or $chosen.End -lt $start -or
        $chosen.End -gt [DateTimeOffset]::UtcNow) { throw 'Metadatos temporales invalidos.' }
    $pattern = '^foodstore_[0-9]{8}T[0-9]{13}Z_[0-9a-f]{32}\.dump$'
    if ($m.file -isnot [string] -or $m.file -cnotmatch $pattern) { throw 'Nombre no permitido.' }
    if (($m.size -isnot [long] -and $m.size -isnot [int]) -or $m.size -le 0 -or
        $m.sha256 -isnot [string] -or $m.sha256 -cnotmatch '^[0-9a-f]{64}$') {
        throw 'Tamano o hash invalido.'
    }
    Assert-Baseline $m.baseline
    $dump = Join-Path $BackupDir $m.file
    Assert-Private $dump
    $hash = (Get-FileHash -LiteralPath $dump -Algorithm SHA256).Hash.ToLowerInvariant()
    if ((Get-Item -LiteralPath $dump).Length -ne $m.size -or $hash -cne $m.sha256) {
        throw 'Respaldo alterado.'
    }
    Pg 'pg_restore' @('--list',$dump) | Out-Null
    [pscustomobject]@{ Manifest=$m; Start=$start; Dump=$dump }
}
```